



KHOA CÔNG NGHỆ THÔNG TIN

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ – ĐẠI HỌC QUỐC GIA HÀ NỘI

**Xây dựng đường cong Elliptic bậc siêu cao
và ứng dụng Chữ ký số**

KHÓA LUẬN TỐT NGHIỆP
NĂM HỌC 2025–2026

Người thực hiện: Trần Mạnh Duy

Giảng viên hướng dẫn: TS. GVC. Lê Phê Đô

Nội dung trình bày

- 01 | Giới thiệu
- 02 | Cơ sở lý thuyết toán học
- 03 | Cài đặt hệ thống
- 04 | Kết luận

ECC là nền tảng bảo mật hạ tầng số

TLS 1.3, SSH, Bitcoin (secp256k1), Ethereum (BLS12-381), chữ ký số điện tử

Thực trạng:

- ▶ Hầu hết hệ thống **tái sử dụng** đường cong sinh sẵn bởi NIST, SECG
- ▶ Tham số dùng “hạt giống” không giải thích được $\Rightarrow$ nguy cơ backdoor
- ▶ Sự kiện Dual_EC_DRBG: NSA bị cáo buộc cài cửa hậu vào PRNG chuẩn NIST
- ▶ Mức bảo mật phổ biến 128-bit — chưa đáp ứng yêu cầu dài hạn NIST 2030+

Đường cong	$ p $	Bảo mật	Minh bạch tham số
secp256k1	256 bit	128-bit	Có
P-256 (NIST)	256 bit	128-bit	Không
BLS12-381	381 bit	128-bit	Có

Hạn chế của các hệ thống hiện tại:

- ▶ Mức bảo mật cố định 128-bit — không đủ cho kịch bản dài hạn
- ▶ Tham số không minh bạch (NIST P-256: seed không giải thích được)
- ▶ Không có thư viện tự chủ, tự kiểm chứng từ đầu bằng ngôn ngữ an toàn bộ nhớ

BLS12-381 được chọn làm baseline kiểm chứng vì tham số minh bạch và đã được cộng đồng quốc tế xác minh rộng rãi.

Vấn đề cần giải quyết

1. Tự chủ mật mã

Xây dựng thư viện ECC tự chủ, không phụ thuộc thư viện mật mã bên ngoài

2. Kiểm chứng minh bạch

Mọi phép toán có thể kiểm chứng độc lập bằng đường cong chuẩn đã biết

Thách thức kỹ thuật

- ▶ Cài đặt số học bignum 1024-bit từ đầu (không dùng thư viện có sẵn)
- ▶ Chứng minh tính đúng đắn toán học cho mọi phép toán
- ▶ Đảm bảo an toàn bộ nhớ tuyệt đối (Rust)
- ▶ Kháng các tấn công đã biết trên đường cong elliptic

Xây dựng thư viện

- ▶ Bignum U1024 tự viết: cộng, trừ, nhân, lũy thừa, nghịch đảo (Rust)
- ▶ Số học Montgomery trên trường hữu hạn $\mathbb{F}_p$
- ▶ Phép toán nhóm trên đường cong Short Weierstrass
- ▶ Phương pháp CM ($D = -3$) kiểm chứng bằng BLS12-381

Ứng dụng & Kiểm chứng

- ▶ Cài đặt hai sơ đồ chữ ký: **Schnorr** và **ECDSA**
- ▶ Bộ kiểm thử toàn diện (36 test cases)
- ▶ Kháng tấn công MOV, Anomalous, Invalid Curve
- ▶ Không phụ thuộc thư viện mật mã bên ngoài

Cơ sở toán học: Trường số nguyên tố $\mathbb{F}_p$

Định nghĩa

$\mathbb{F}_p = \{0, 1, \dots, p-1\}$ với phép cộng và nhân $(\text{mod } p)$,
mọi phần tử $a \neq 0$ có nghịch đảo $a^{-1} = a^{p-2} \pmod{p}$

Tối ưu phép nhân: Dạng biểu diễn Montgomery

- ▶ Chọn $R = 2^{1024}$, biểu diễn $\hat{a} = a \cdot R \pmod{p}$
- ▶ Phép nhân Montgomery (REDC):
 $\hat{a} \cdot \hat{b} \cdot R^{-1} \pmod{p}$
- ▶ Thay phép chia cho p bằng
phép chia cho R (dịch bit)

	Thông thường	Montgomery
Rút gọn	$\div p$	$\div R$ (dịch bit)
Chi phí	$O(n^2) + \text{chia}$	$O(n^2)$

Short Weierstrass trên $\mathbb{F}_p$

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

Phép cộng điểm $P_1 + P_2 = P_3$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2$$

Phép nhân vô hướng:

$[k]P$ bằng thuật toán *Double-and-Add*,
độ phức tạp $O(\log k)$

Bài toán logarit rời rạc (ECDLP)

- ▶ Cho $P, Q = [k]P$, tìm k là bài toán khó (cơ sở bảo mật)
- ▶ Độ phức tạp: $O(\sqrt{r})$ với thuật toán Pollard's rho
- ▶ Nhóm con cấp r nguyên tố đảm bảo an toàn

Điều kiện CM: Tồn tại đường cong trên $\mathbb{F}_p$ với vết Frobenius t khi:

$$4p = t^2 + |D| \cdot y^2 \quad (D < 0 \text{ là biệt thức CM})$$

Quy trình 4 bước:

1. Tính j -invariant từ đa thức lớp Hilbert $H_D(x) \pmod{p}$
2. Suy ra hệ số đường cong (a, b) từ j
3. Kiểm tra xoắn (twist test): chọn đường cong có $r \mid \#E$
4. Tìm điểm sinh G bậc r bằng nhân cofactor

Schnorr — 384 byte

Ký: $k \xleftarrow{R} [1, r-1]$, $R = [k]G$

$e = H(R_x \| R_y \| m) \bmod r$

$s = k + e \cdot d \pmod{r}$

Chữ ký: $\sigma = (R, s)$

Xác minh: $[s]G \stackrel{?}{=} R + [e]Q$

ECDSA (FIPS 186-4) — 256 byte

Ký: $k \xleftarrow{R} [1, r-1]$, $R = [k]G$

$r_{\text{sig}} = R_x \bmod n$, $e = H(m)$

$s = k^{-1}(e + r_{\text{sig}} \cdot d) \bmod n$

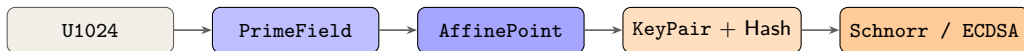
Chữ ký: (r_{sig}, s)

Xác minh: $[u_1]G + [u_2]Q$, kiểm P_x

03 | Cài đặt hệ thống



Công cụ dòng lệnh curve1024-sig



Lệnh CLI

```
keygen — sinh cặp khóa  
sign -f <file> -scheme  
schnorr|ecdsa  
verify -f <file> -k <pub>  
export-pub — xuất khóa công khai
```

Đặc điểm

- ▶ Tham số đường cong cấu hình qua file TOML
- ▶ Không phụ thuộc thư viện mật mã bên ngoài
- ▶ Hỗ trợ thay đổi đường cong mà không sửa mã nguồn

03 | Cài đặt hệ thống

So sánh hiệu năng



Đường cong	$ p $	Ký	Xác minh	Bảo mật
secp256k1	256	~ 0.2 ms	~ 0.4 ms	128-bit
P-384	384	~ 1 ms	~ 2 ms	192-bit
BLS12-381	381	~ 1.5 ms	~ 3 ms	128-bit
Curve1024*	381	1.55 s	2.83 s	256-bit**

Schnorr sign/verify — Criterion, release build, LLVM O3.
Do trên phần mềm thuần Rust, chưa tối ưu ASM.

* Chạy trên BLS12-381 (381-bit) với kiến trúc U1024.

** Kiến trúc U1024 hỗ trợ bảo mật 256-bit khi dùng tham số 1024-bit thực thụ.

Nguyên nhân chậm:

- ▶ U1024 luôn xử lý đủ 16 limb bất kể p thực tế $\Rightarrow$ chi phí tương đương trường 1024-bit
- ▶ Chưa có Montgomery Ladder, tọa độ Projective, inline ASM

Kịch bản phù hợp:

- ▶ Ký tài liệu pháp lý, phần mềm
- ▶ Chứng chỉ PKI dài hạn
- ▶ Hệ thống cần bảo mật hậu lượng tử mức 256-bit

Kết quả

- ✓ Kiến trúc bignum U1024, trường nguyên tố và các phép toán
- ✓ Số học Montgomery, phép toán nhóm Short Weierstrass đúng đắn
- ✓ Phương pháp CM ($D = -3$), kiểm chứng chéo BLS12-381
- ✓ Schnorr & ECDSA hoạt động đúng
- ✓ Có thể kháng tấn công Anomalous ($\#E \neq p$) và Invalid Curve

Hạn chế

- ▶ **Timing side-channel**
Double-and-Add rò rỉ bit scalar
⇒ cần Montgomery Ladder
- ▶ **Tọa độ Affine**
1 nghịch đảo/phép cộng điểm
⇒ cần Jacobian/Projective
- ▶ **Hiệu năng**
~8000× chậm hơn secp256k1
(trường lớn $4\times$ + chưa tối ưu ASM)

Hướng phát triển

1. Tối ưu hiệu năng

Tọa độ Projective	1 nghịch đảo/lần nhân vô hướng (dự kiến $\sim 10\times$)
Montgomery Ladder	Constant-time — loại bỏ timing side-channel
Sliding Window	Giảm $\sim 25\%$ số phép cộng điểm

⇒ Thu hẹp khoảng cách hiệu năng với secp256k1 từ $8000\times$ xuống $\sim 500\times$

2. Tích hợp tham số 1024-bit thực thụ

Hoàn thiện quy trình CM sinh tham số trên trường 1024-bit và tích hợp vào `curve1024.toml`

⇒ Khai thác đầy đủ kiến trúc U1024 — mục tiêu bảo mật 256-bit đối xứng

3. Kiểm định hình thức

Chứng minh tính đúng đắn bằng Creusot/Kani cho Rust

⇒ Đảm bảo mức tin cậy cần thiết cho triển khai mật mã thực tế

Cảm ơn thầy cô đã lắng nghe!